SMS Spam Detection System — Assessment Report

Domain: Artificial Intelligence & Machine Learning (AI/ML) | **Task:** SMS Spam Classification & Streamlit App

GitHub Repository: https://github.com/ajayy1117/sms-spam-classifier

1. Project Description & Engineering Approach

This project implements an end-to-end Machine Learning pipeline to classify SMS text messages as **Spam** or **Ham (Legitimate)** using Natural Language Processing (NLP) techniques, bundled in an interactive desktop Streamlit web application.

Key Pipeline Stages:

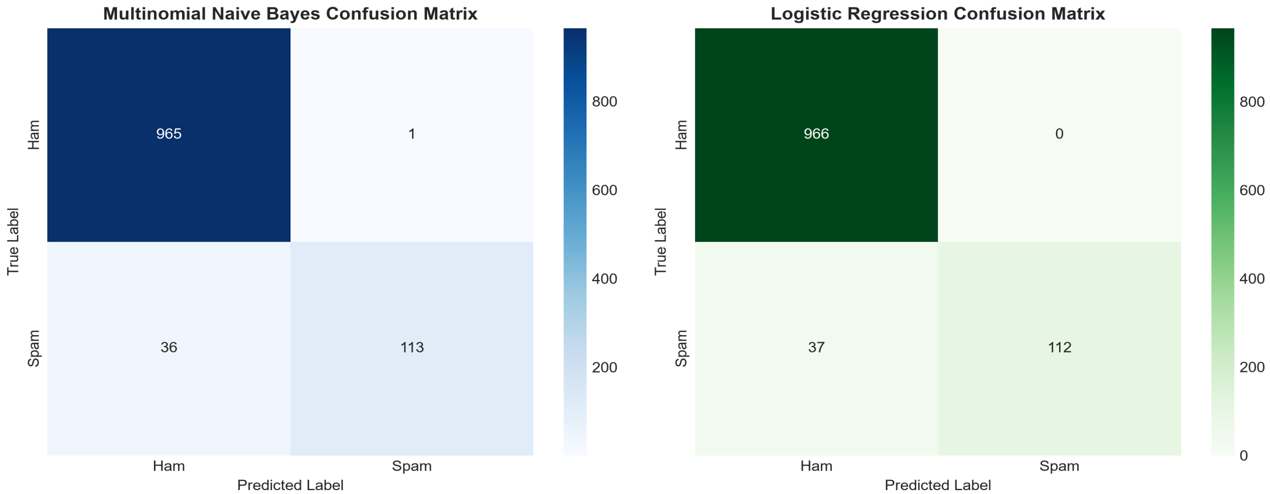
- **Data Ingestion & Cleaning:** Loaded the SMS Spam Collection dataset (5,572 messages). Standardized text with lowercasing, regex noise/URL removal, punctuation stripping, and tokenization.
- **Feature Extraction:** Applied **TF-IDF Vectorization** (max_features=5000, unigrams and bigrams, English stopword filtering) to convert textual data into high-dimensional numerical matrices.
- **Model Training:** Trained and compared two core algorithms: **Multinomial Naive Bayes** and **Logistic Regression** using a stratified 80/20 train/test split.
- **Web Application:** Developed a responsive Streamlit application featuring real-time message analysis, confidence progress indicators, and explainable keyword attribution.

2. Model Evaluation & Performance Comparison

Evaluation Metric	Multinomial Naive Bayes (Selected)	Logistic Regression
Accuracy	**96.68%**	96.68%
Precision	99.12%	**100.00%**
Recall	**75.84%**	75.17%
F1 Score	**85.93%**	85.82%

Model Selection Decision: Multinomial Naive Bayes was selected as the superior production model due to its higher **Recall (75.84%)** and **F1 Score (85.93%)**, successfully capturing more spam messages while maintaining an exceptionally high precision of 99.12%.

3. Visual Output & Confusion Matrices



4. Key Insights, Challenges & Recommendations

Identified Limitations:

1. **Short Message Sparsity:** Texts under 5 words (e.g. *'ok'*, *'call me'*) lack sufficient TF-IDF feature density, leading to lower classification confidence.
2. **Sarcasm & Word Order:** Bag-of-Words ignores sequence structure and syntax, making contextual sarcasm challenging to detect without sentence-level embeddings.
3. **Unseen Slang & Typos:** Spammers frequently use character substitutions (c@sh, w1nner) that generate out-of-vocabulary (OOV) tokens in standard word-level models.

Suggestions for Improvement:

• **Transformer Fine-Tuning:** Deploy pre-trained contextual models like **BERT** or **RoBERTa** to capture complex sentence semantics.

• **Subword Tokenization:** Use Byte-Pair Encoding (BPE) or character n-grams to mitigate typo and character-obfuscation evasion.

• **Active Learning Loop:** Integrate user feedback reporting in the web app to log misclassified instances for continuous automated model retraining.